

Aula 2 - Conceitos Básicos

Prof. Danilo de Quadros Maia Filho

Departamento:
Engenharia de Software

Sumário

- 1 Medição
- 2 Escopo das Métricas de Software
- 3 Métricas Modernas: Ágil, DevOps e IA
- 4 Normas Internacionais
- 5 Artigos Recomendados

Sumário

- 1 Medição
 - Relevância
 - Objetivos da Medição de Software
 - Goal-Question-Metric (GQM)
- 2 Escopo das Métricas de Software
- 3 Métricas Modernas: Ágil, DevOps e IA
- 4 Normas Internacionais
- 5 Artigos Recomendados

Seria difícil imaginar as engenharias Civil, Elétrica ou Mecânica sem o papel central da medição. Seja na ciência ou engenharia não seriam práticas nem efetivas sem medição.

No entanto, muitas vezes e em alguns projetos de software, a Medição é considerada um luxo!



Figura 1: Gráfico de Gantt no Jira

- **Requisitos não-funcionais:** Prometemos que o sistema será user-friendly, confiável e sustentável sem especificar claramente e objetivamente o que isso significa. Dessa forma, não temos um objetivo bem definido a ser alcançado.
- **Custos:** Falhamos em entender e quantificar os custos dos projetos de software. Custos de design, codificação, testes. Medir os atributos que modelam o custo é fundamental.
- **Novas ferramentas:** Aceitamos inovação de ferramentas disruptivas apenas pela promessa, sem avaliar de forma criteriosa, sistemática e científica se a nova proposta, nova técnica ou nova ferramenta se propõem, com alguma probabilidade, a aumentar a eficiência.
- **Argumentação:** Aceitamos argumentações sem evidência clara.

Um mito muito citado (e errado)

A frase “*você não pode gerenciar o que não consegue medir*” é constantemente atribuída a W. Edwards Deming, um dos fundadores da gestão da qualidade moderna. O problema é que Deming disse, quase literalmente, o oposto.

Em *The New Economics* (3ª ed., p. 26), Deming escreveu: “*É errado supor que, se você não pode medir algo, não pode gerenciá-lo — um mito custoso*”. Em *Out of the Crisis* (p. 121), ele reforça que “*as figuras mais importantes que alguém precisa para a gestão são desconhecidas e incognoscíveis*”. Gerenciar apenas por números visíveis é, para Deming, uma das “sete doenças mortais da gestão”.

A lição não é abandonar a medição — é usá-la com humildade, ciente de que nem tudo o que importa cabe em um indicador.

Também é comum não avaliarmos a procedência das informações. De onde o dado veio? Como foi o processo de medição? Qual a margem de erro? Que garantias temos a respeito da qualidade, acurácia, abrangência, universo amostral? Quais as referências para comparação?

Como dizer se um projeto está ou não saudável? Quais os critérios? O que deveria ser o alvo da medição? Projeto, Produto, Processos, Recursos.

Objetivos da Medição de Software

A medição provê feedback para o objetivo proposto. Portanto, antes de definir a medição a ser feita, é importante definir os objetivos que se deseja alcançar.

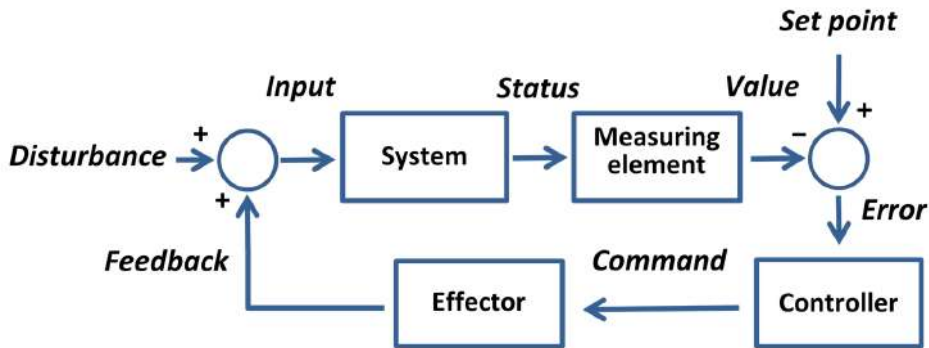


Figura 2: Sistema com Feedback

Objetivos da Medição de Software

Stakeholders:

- Managers
- Developers

Objetivos da Medição de Software

Managers:

- 1 Qual o custo de cada processo? Seria possível otimizar os processos de forma a minimizar custos? Qual o custo de se propagar conhecimento dos processos junto à equipe?
- 2 Quão produtiva é a equipe? Existe alguma estruturação da equipe que maximize a eficiência? Qual o nível de senioridade da equipe e como o conhecimento está distribuído? Como lidar com a rotatividade dos colaboradores?
- 3 Quão bom é o código sendo desenvolvido? Existem alternativas? Como comparar o status atual do código a possíveis tomadas de decisão? Seria possível propor mudanças de arquitetura com base em alguma informação?
- 4 O usuário estará satisfeito com as entregas? Qual o histórico de satisfação? Qual o perfil dos usuários? O produto agrada a todos? Existem nichos? As funcionalidades e processos devem ser segmentados?
- 5 Como e onde podemos aprimorar? Quando alterar a rota? Mudanças são bem vindas? Como justificar a melhoria? O quanto estamos aprendendo e como está sendo trabalhada a gestão do conhecimento?

Objetivos da Medição de Software

Developers:

- 1 Os requisitos são testáveis? Qual a qualidade dos requisitos? Os requisitos batem com o que foi implementado? Qual a parcela do time compreendeu os requisitos de acordo com o objetivo? Os requisitos e critérios de aceite correspondem?
- 2 Nós encontramos todas as falhas? É possível prever falhas? Se não as falhas propriamente ditas, os volumes? Qual a proporção de falhas que já foi encontrada comparada com a quantidade de falhas previstas?
- 3 Atingimos os objetivos do produto? E dos processos exigidos para que o software seja funcional? Qual porcentagem dos objetivos foi cumprida? Como correlacionar o que foi de fato entregue aos objetivos?
- 4 O que acontecerá no futuro? É possível prever baseado no presente? As informações históricas podem ser utilizadas cumulativamente para prever tendências do código?

Goal-Question-Metric (GQM)

Vimos que managers e developers têm perguntas muito diferentes. Como transformar essas perguntas em métricas de forma sistemática, e não por tentativa e erro?

O **GQM (Goal-Question-Metric)**, proposto por Victor Basili e colegas na Universidade de Maryland/NASA-SEL no final dos anos 1980, é o framework mais influente para isso: parte-se de um **objetivo** de negócio, derivam-se **perguntas** que indicam se o objetivo está sendo alcançado, e só então definem-se as **métricas** que respondem a cada pergunta.

A ideia central: *nunca colete uma métrica sem antes saber qual pergunta ela responde e a que objetivo essa pergunta serve.*

Goal-Question-Metric (GQM)

Template do objetivo (Basili, Caldiera & Rombach, 1994):

Analisar o [objeto de estudo]
com o propósito de [propósito, ex.: caracterizar, avaliar, prever, melhorar]
com respeito a [foco de qualidade, ex.: custo, confiabilidade, satisfação do usuário]
do ponto de vista de [quem usa a informação, ex.: gerente, desenvolvedor]
no contexto de [ambiente, projeto, organização]

Exemplo: *Analisar o processo de revisão de código com o propósito de melhorar, com respeito ao tempo de ciclo, do ponto de vista do gerente de engenharia, no contexto do time de pagamentos.*

Goal-Question-Metric (GQM)

Exemplo aplicado:

- **Goal:** melhorar a previsibilidade das entregas do time.
- **Question 1:** quanto tempo, em média, uma tarefa leva do início ao fim?
 - **Metric:** tempo de ciclo (*cycle time*) por tarefa.
- **Question 2:** o time está estimando de forma consistente?
 - **Metric:** desvio entre esforço estimado e esforço realizado.
- **Question 3:** onde as tarefas ficam mais tempo paradas?
 - **Metric:** tempo médio por etapa do quadro Kanban.

Note: cada métrica só existe porque responde a uma pergunta — nunca o contrário.

GQM+Strategies — a evolução moderna

Uma crítica ao GQM clássico: ele conecta objetivos a métricas dentro de **um único nível** (um time, um projeto). Mas como garantir que as métricas de um time realmente contribuem para os objetivos **estratégicos** da organização?

O **GQM+Strategies** (Basili et al., 2010) estende o modelo criando uma cadeia de objetivos e métricas que atravessa múltiplos níveis — do time de desenvolvimento até a diretoria — tornando explícito *por que* cada métrica de equipe importa para o negócio.

Referência: BASILI, V. et al. *Aligning Organizations Through Measurement: The GQM+Strategies Approach*. Springer, 2014.

Sumário

1 Medição

2 Escopo das Métricas de Software

- Estimativa de custo e esforço
- Coleta de Dados
- Modelos e medidas de qualidade
- Modelos de Confiabilidade
- Métricas de Segurança
- Métricas estruturais e de complexidade
- Avaliação de maturidade de capacidade
- Gestão por Métricas

3 Métricas Modernas: Ágil, DevOps e IA

4 Normas Internacionais

5 Artigos Recomendados

Escopo das Métricas de Software

Atividades em engenharia de software que envolvem algum nível de medição:

- Modelos e medidas de estimativa de custo e esforço
- Coleta de dados
- Modelos e medidas de qualidade
- Modelos de confiabilidade
- Métricas de segurança
- Métricas estruturais e de complexidade
- Avaliação de maturidade de capacidade
- Gestão por métricas

Na sequência, veremos ainda um bloco dedicado a métricas mais recentes — ágeis, DevOps, observabilidade e Inteligência Artificial — na seção *Métricas Modernas*.

Estimativa de custo e esforço

Gerentes de projetos foram os principais motivadores para o desenvolvimento e uso de métricas de software, com o objetivo de prever custos ainda nas fases iniciais do ciclo de vida do projeto. Para isso, surgiram diversos modelos de estimativa de custo e esforço, como o COCOMO II de Boehm e o modelo de pontos de função de Albrecht. Esses modelos geralmente calculam o esforço como uma função pré-definida de variáveis como o tamanho do software (em linhas de código ou pontos de função), a capacidade da equipe de desenvolvimento e o nível de reutilização de componentes.

Estimativa de custo e esforço — visão ágil

Times ágeis, em geral, não usam COCOMO ou pontos de função diretamente. Em vez de estimar em horas logo no início, eles costumam:

- Estimar em unidades relativas, como **story points** (via Planning Poker), comparando o tamanho relativo das tarefas em vez do esforço absoluto;
- Usar a **velocidade** (velocity) do time — quantos pontos foram entregues nos últimos sprints — para projetar entregas futuras;
- Reestimar continuamente, em vez de travar uma estimativa única no início do projeto.

Um movimento minoritário, mas influente, o **#NoEstimates**, questiona até a utilidade de estimar: defende medir o fluxo de trabalho real (quantos itens terminam por semana) em vez de prever esforço antes de começar.

Coleta de Dados

A qualidade de um programa de medição depende diretamente da coleta cuidadosa de dados, o que é especialmente desafiador quando se trata de projetos diversos. Por isso, a coleta de dados tem se tornado uma disciplina própria, com especialistas focados em garantir definições claras das métricas, consistência no processo, completude e integridade dos dados. Esse processo precisa ser planejado e conduzido com atenção e sensibilidade.

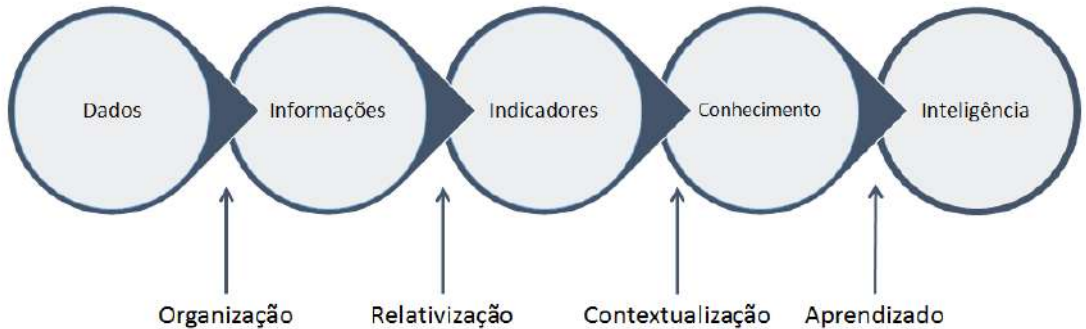


Figura 3: Dado à Inteligência

<http://teianconsultoria.com.br/gerenciamento-por-indicadores/>

Tipos de informações nas Organizações

DADOS	INFORMAÇÕES	INDICADORES
Disponível para manipulação no banco de dados.	Organizados e já manipulados em primeiro nível.	Manipulados matematicamente através de fórmulas.
Abundantes e armazenados em sua totalidade.	Selecionados em formatos de telas e/ou relatórios.	Parametrizados em formatos de gráficos lineares.
Viabilizados através de coleta de dados.	Viabilizado através de softwares gerenciais.	Viabilizados através de regras de contagem.
Não tem foco na gestão.	Com foco abrangente e dispersivo.	Com foco no que é relevante.

Metodologia considera o conceito de Driver e Outcome

OUTCOME (Fim) - Gerenciamento pelo RESULTADO

Dependem dos driver para serem operacionalizados.

Envolvem ações de vários cargos de forma compartilhada para sua operacionalização efetiva.

DRIVER (Meio) - Gerenciamento pelo ESFORÇO

Traduzem os indicadores outcomes em ações individualizadas.

Envolvem ações de um único cargo para sua operacionalização efetiva.

Figura 4: Dado, Informação e Indicador

Modelos e medidas de qualidade

Como a qualidade de software envolve diversos fatores, engenheiros de software desenvolveram modelos que representam a interação entre esses fatores. Esses modelos geralmente têm estrutura em forma de árvore, em que os ramos superiores contêm fatores de qualidade de alto nível — como confiabilidade e usabilidade — que se deseja quantificar. Cada fator de qualidade é dividido em critérios de nível inferior, como modularidade e compartilhamento de dados, que são mais fáceis de entender e medir. As métricas são aplicadas a esses critérios, e a árvore descreve as relações entre os fatores e seus critérios dependentes, permitindo medir os fatores com base nas métricas dos critérios. Essa abordagem de “dividir para conquistar” tornou-se padrão na medição da qualidade de software, formalizada pela metodologia da norma IEEE 1061 (ver seção *Normas Internacionais*).

Modelos e medidas de qualidade

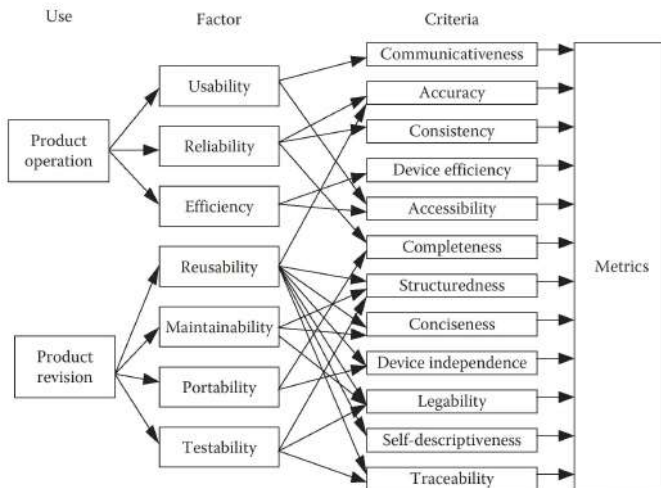


Figura 5: Modelo de Qualidade

O modelo de qualidade vigente: ISO/IEC 25010

O modelo em árvore mais usado hoje não é mais o antigo ISO/IEC 9126 — é a **ISO/IEC 25010**, parte da família SQuaRE (*Software Product Quality Requirements and Evaluation*). A versão **2023** trouxe uma atualização importante, ampliando de 8 para **9 características**:

Característica	O que avalia
Adequação funcional	O software faz o que deveria fazer?
Eficiência de desempenho	Uso de tempo e recursos (CPU, memória, rede).
Compatibilidade	Coexiste e troca dados com outros sistemas.
Capacidade de interação (<i>novo nome</i>)	Ex- "usabilidade" — inclui agora inclusividade e autodescrição.
Confiabilidade	Mantém o desempenho sob condições esperadas.
Segurança	Protege dados e mantém níveis de acesso apropriados.
Manutenibilidade	Facilidade de modificar, corrigir e testar.
Flexibilidade (<i>novo nome</i>)	Ex- "portabilidade" — inclui agora escalabilidade.
Segurança física (Safety) (<i>nova!</i>)	Evita danos a pessoas, propriedade ou ambiente.

Por que a ISO 25010:2023 importa hoje

- A entrada de **Safety** como característica própria reflete a preocupação crescente com software embarcado em carros autônomos, dispositivos médicos e sistemas industriais.
- **Capacidade de Interação** formaliza *inclusividade* e *acessibilidade* como parte da qualidade — não mais um “extra”.
- **Flexibilidade** (antiga portabilidade) passa a considerar explicitamente *escalabilidade*, essencial para arquiteturas em nuvem e microsserviços.

Referência: ISO/IEC 25010:2023 — *Systems and software engineering — SQaRE — Product quality model*.

Modelos de Confiabilidade

A maioria dos modelos de qualidade inclui a confiabilidade como um de seus fatores componentes. No entanto, a necessidade de prever e medir a confiabilidade com mais precisão levou ao surgimento de uma especialização própria em modelagem e previsão de confiabilidade.

Métricas clássicas:

- **MTTF** (*Mean Time To Failure*): tempo médio até a primeira falha;
- **MTBF** (*Mean Time Between Failures*): tempo médio entre falhas consecutivas em sistemas reparáveis;
- **MTTR** (*Mean Time To Repair/Restore*): tempo médio para restaurar o serviço após uma falha;
- **Densidade de defeitos**: número de defeitos por KLOC (mil linhas de código) ou por ponto de função.

Modelos de Confiabilidade

Modelos clássicos de **crescimento de confiabilidade** (*software reliability growth models*) tentam prever, a partir do histórico de falhas em teste, quantos defeitos ainda restam e quando o software estará “confiável o suficiente” para ser liberado:

- **Modelo de Musa** (*Basic Execution Time Model*, 1975): relaciona a taxa de falhas ao tempo de execução do software.
- **Modelo de Goel–Okumoto** (1979): usa um processo de Poisson não-homogêneo para modelar a chegada de falhas ao longo do tempo.

Tendência moderna — Engenharia do Caos: em vez de só prever falhas por modelos estatísticos, empresas como a Netflix (com o *Chaos Monkey*, 2011) passaram a *injetar falhas propositalmente* em produção para medir, empiricamente, a resiliência real do sistema antes que uma falha real aconteça.

Com a computação integrada a quase todas as atividades humanas, aumentaram as preocupações com a segurança dos sistemas de software. Há receio de que invasores roubem ou corrompam arquivos, senhas e contas. A segurança depende tanto do design interno do sistema quanto do tipo de ataque externo que ele pode sofrer.

Métricas de Segurança — panorama atual

- **CVSS** (*Common Vulnerability Scoring System*): padrão da indústria (mantido pelo FIRST) para pontuar a severidade de uma vulnerabilidade de 0 a 10, considerando explorabilidade e impacto.
- **OWASP Top 10**: ranking, atualizado periodicamente, das categorias de vulnerabilidade mais críticas em aplicações web — usado como referência de benchmark e checklist de avaliação.
- **SBOM** (*Software Bill of Materials*): inventário formal de todas as dependências de um software, hoje exigido em contratos governamentais dos EUA após incidentes de cadeia de suprimentos como o *Log4Shell* (2021) e o ataque à *SolarWinds* (2020).
- **Métricas de DevSecOps**: tempo médio para remediar uma vulnerabilidade (MTTR de segurança), % de dependências desatualizadas, cobertura de *SAST/DAST* (análise estática/dinâmica de segurança) no pipeline de CI/CD.

Métricas estruturais e de complexidade

Atributos de qualidade desejáveis, como confiabilidade e manutenibilidade, geralmente só podem ser medidos após a existência de uma versão operacional do código. No entanto, é importante prever, ainda antes da conclusão do sistema, quais partes podem ser menos confiáveis, mais difíceis de testar ou manter. Para isso, medem-se atributos estruturais de representações do software disponíveis antecipadamente, sem necessidade de execução. Com base nessas medições, desenvolvem-se teorias preditivas empíricas que apoiam a garantia, o controle e a previsão da qualidade. Essas representações incluem gráficos de fluxo de controle (modelando o código) e diagramas da UML (modelando o design e os requisitos). As métricas estruturais também consideram aspectos como a organização dos módulos e o uso de padrões de projeto.

Métricas estruturais clássicas — nomeando os principais atores

- **Complexidade Ciclômática** (McCabe, 1976): conta os caminhos independentes em um grafo de fluxo de controle; valores altos indicam código difícil de testar e manter.
- **Métricas de Halstead** (1977): estimam esforço e volume a partir da contagem de operadores e operandos distintos no código.
- **Suíte de Chidamber e Kemerer** (CK, 1994) — para orientação a objetos:
 - WMC (*Weighted Methods per Class*), DIT (profundidade da árvore de herança), NOC (número de filhos), CBO (acoplamento entre objetos), RFC (*Response For a Class*), LCOM (falta de coesão dos métodos).
- **Índice de Manutenibilidade** (*Maintainability Index*): combina complexidade ciclômática, volume de Halstead e linhas de código em um único indicador de 0 a 100.

Métricas estruturais na prática

Hoje, essas métricas raramente são calculadas à mão — ferramentas de **análise estática** as coletam automaticamente a cada commit:

- **SonarQube / SonarCloud**: calcula complexidade, duplicação, cobertura de testes e *Technical Debt Ratio* (baseado no modelo SQALE), bloqueando merges que não atingem um *Quality Gate* mínimo.
- **CodeClimate, ESLint, Pylint, Checkstyle**: análise estática específica por linguagem, muitas vezes integrada diretamente ao pipeline de CI/CD.

Para refletir

Se uma ferramenta bloqueia o merge quando a complexidade ciclomática ultrapassa 10, o time passa a otimizar para “ficar abaixo de 10” — e não necessariamente para escrever o código mais simples possível. É o mesmo risco que veremos adiante com a Lei de Goodhart.

Avaliação de maturidade de capacidade

O modelo Capability Maturity Model Integration (CMMI®) for Development é uma técnica de avaliação de processos de desenvolvimento de software, originalmente desenvolvida pelo **Software Engineering Institute (SEI)** da Universidade Carnegie Mellon. Inicialmente criado para medir a capacidade de contratados do governo dos EUA em desenvolver software de qualidade, o CMMI hoje é utilizado por diversas organizações ao redor do mundo. A avaliação considera aspectos como uso de ferramentas, práticas padronizadas e outros fatores de desenvolvimento. Ela envolve a análise de centenas de questões sobre as práticas da organização, resultando em uma classificação em uma escala de cinco níveis, que vai do Nível 1 (Inicial – dependente de indivíduos) ao Nível 5 (Otimizado – processos otimizados com base em dados quantitativos). Atualmente, é comum que empresas exijam certificações CMMI em níveis específicos para contratar fornecedores.

Quem mantém o CMMI hoje?

Atualização de governança

O SEI transferiu a gestão comercial do CMMI para o **CMMI Institute** em 2012, que foi **adquirido pela ISACA** em 2016. Desde então, é a **ISACA** quem desenvolve o modelo e certifica os *Lead Appraisers* — não mais o SEI.

Em **abril de 2023**, a ISACA lançou o **CMMI V3.0**, que:

- mantém os 5 níveis de maturidade, mas simplifica a estrutura de práticas;
- amplia o escopo para além de desenvolvimento de software, incluindo domínios como segurança, gestão de dados, gestão de pessoas e trabalho remoto/virtual;
- integra-se mais diretamente a frameworks ágeis, em vez de ser percebido como “anti-ágil”.

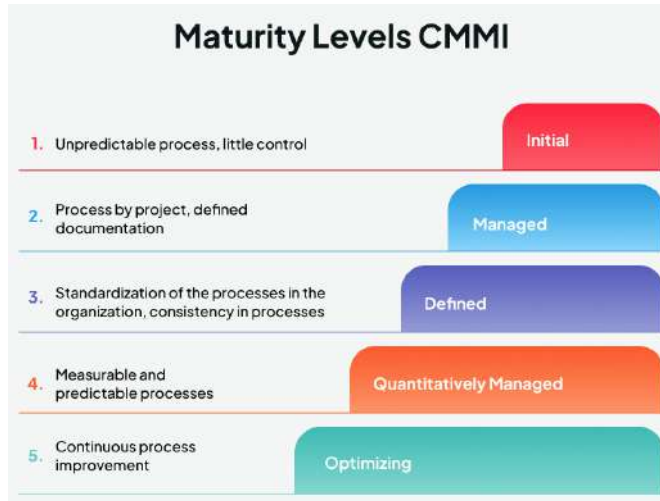


Figura 6: Níveis de Maturidade

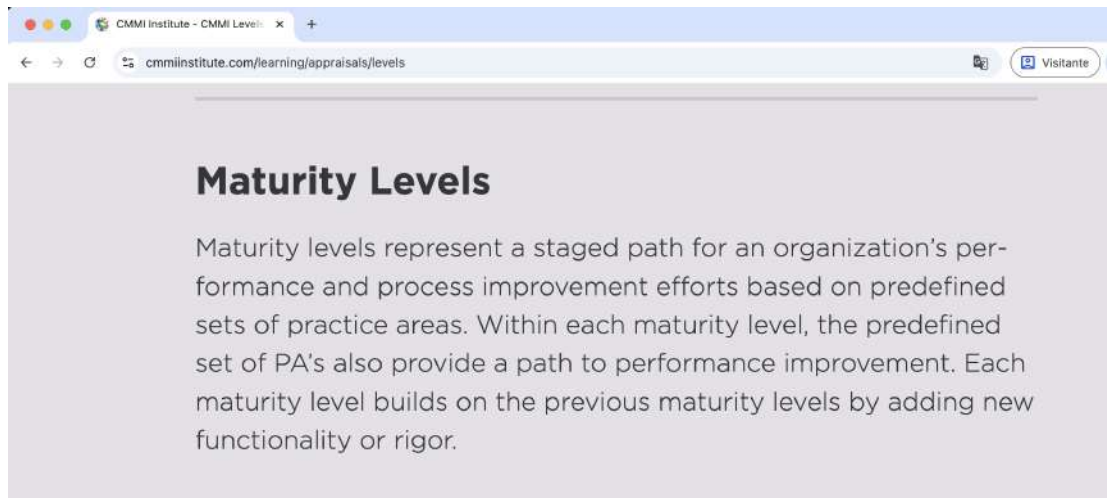


Figura 7: Níveis de Maturidade - site

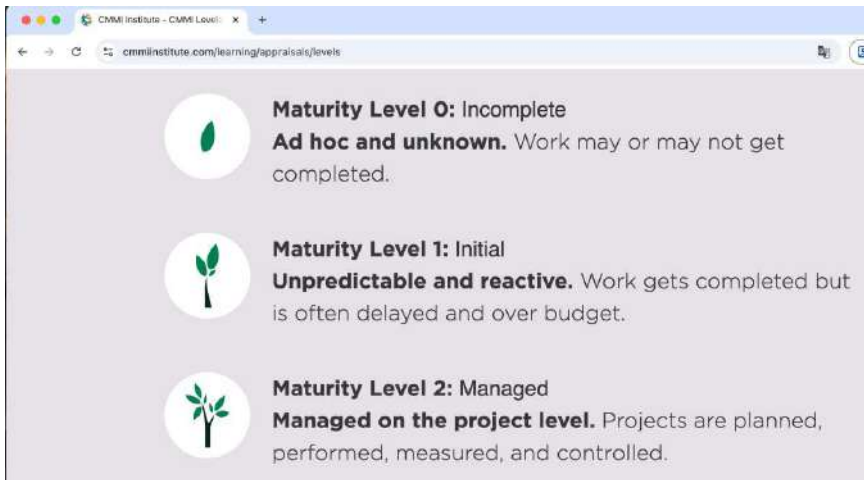


Figura 8: Níveis de Maturidade - site

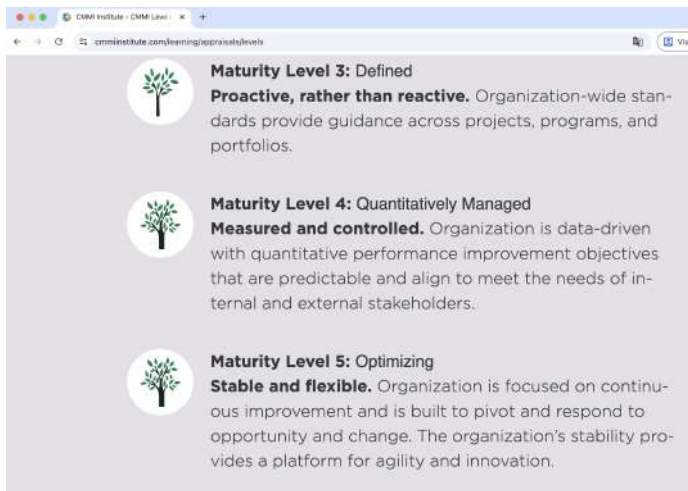


Figura 9: Níveis de Maturidade - site

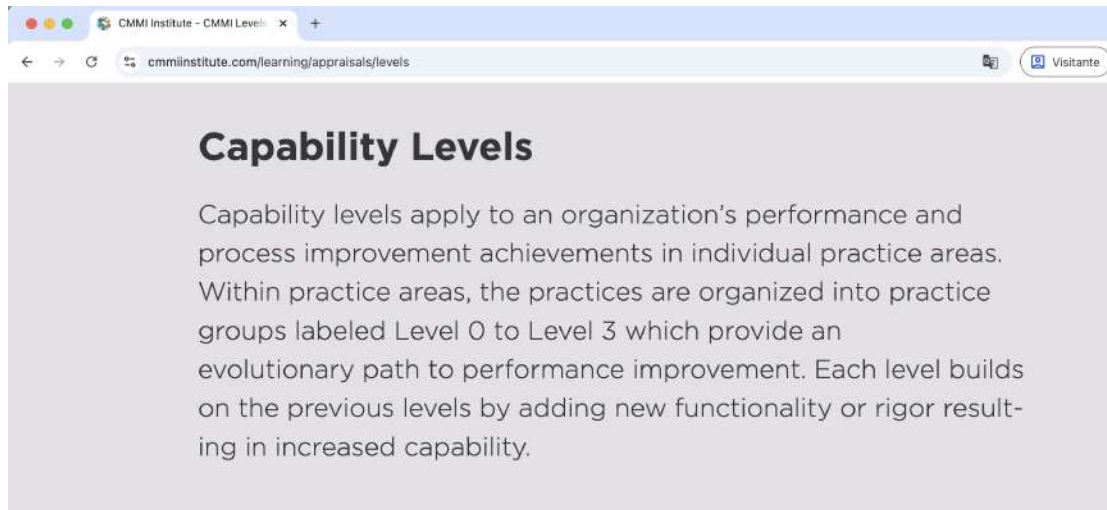


Figura 10: Níveis de Capacidade - site

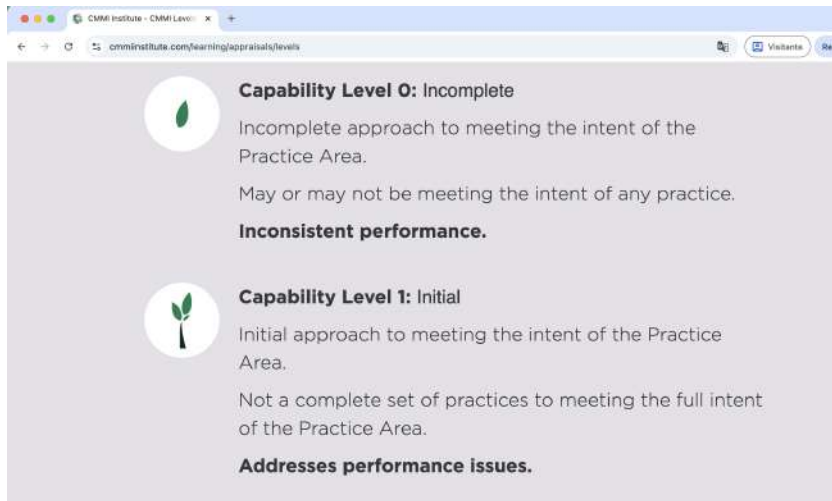


Figura 11: Níveis de Capacidade - site

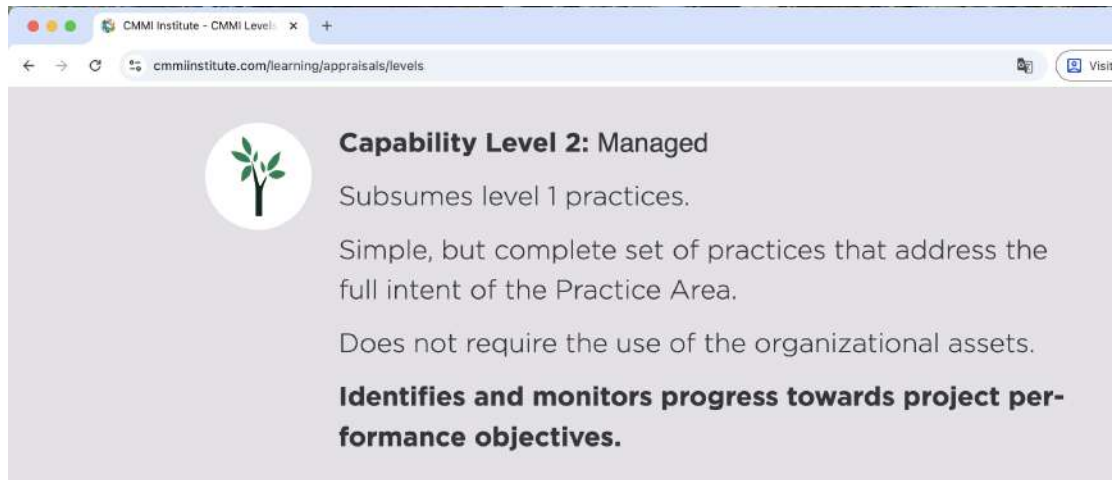


Figura 12: Níveis de Capacidade - site

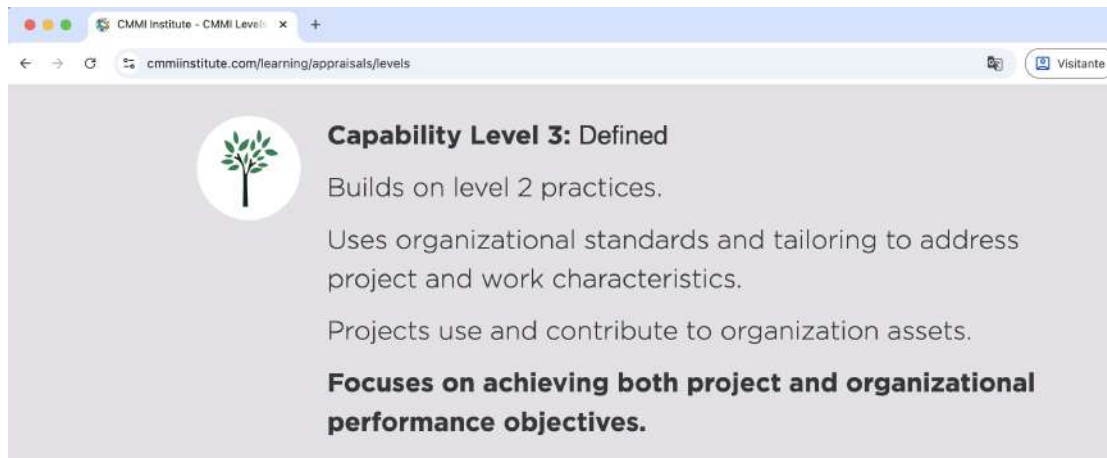


Figura 13: Níveis de Capacidade - site

Gestão por Métricas

A medição é uma parte fundamental da gestão de projetos de software. Tanto clientes quanto desenvolvedores dependem de gráficos e relatórios baseados em métricas para avaliar se o projeto está no caminho certo. Muitas organizações definem conjuntos padronizados de medições e formas de apresentação dos dados para permitir comparações entre projetos. Essa padronização é especialmente importante quando o software é parte de um produto cujo foco principal está fora da área de TI, pois o cliente final geralmente não domina a terminologia técnica. Assim, as métricas ajudam a comunicar o progresso de forma clara e acessível. Por exemplo, um engenheiro responsável por uma usina de energia pode contratar software de controle, mesmo sem ter conhecimento em programação ou testes — nesse caso, as medições ajudam a fornecer uma visão compreensível da evolução do projeto.

8 Metrics to Measure Project Success



Schedule
Variance



Cost Variance



Schedule
Performance
Index



Cost Performance
Index



Quality Metrics



Scope Metrics



Risk Metrics



Stakeholder
Metrics

Figura 14: Métricas de Projeto

Sumário

1 Medição

2 Escopo das Métricas de Software

3 Métricas Modernas: Ágil, DevOps e IA

- Métricas Ágeis e DevOps
- Observabilidade e Site Reliability Engineering
- Quando a métrica vira alvo
- Avaliação de Métodos e Ferramentas
- Métricas e Inteligência Artificial
- Atividade em sala

4 Normas Internacionais

5 Artigos Recomendados

Métricas Modernas: Ágil, DevOps e IA

Os tópicos anteriores formam a base clássica da disciplina de medição de software. Esta seção reúne o que mudou nas últimas duas décadas — sobretudo com a adoção de métodos ágeis, práticas de DevOps/SRE e, mais recentemente, IA generativa:

- Métricas ágeis e DevOps (incluindo as DORA Metrics)
- Observabilidade e Site Reliability Engineering
- Quando a métrica vira alvo: os limites da medição
- Avaliação de métodos e ferramentas
- Métricas e Inteligência Artificial

Métricas Ágeis e DevOps

Times ágeis que trabalham em Scrum ou Kanban usam métricas próprias, focadas em **fluxo de trabalho** mais do que em previsão de custo total:

- **Velocity**: pontos de história entregues por sprint.
- **Lead time** vs. **Cycle time**: tempo desde a solicitação até a entrega vs. tempo desde o início do trabalho até a entrega.
- **Cumulative Flow Diagram (CFD)**: visualiza gargalos mostrando o volume de itens em cada coluna do quadro ao longo do tempo.
- **WIP (Work In Progress)**: itens em andamento simultaneamente — limitar o WIP é um princípio central do Kanban.
- **Burndown / Burnup charts**: trabalho restante (ou concluído) ao longo do sprint/release.

DORA Metrics — o novo padrão da indústria

A pesquisa *DevOps Research and Assessment* (DORA), do Google, consolidada no livro *Accelerate* (Forsgren, Humble & Kim, 2018) e no *State of DevOps Report* anual, identificou **quatro métricas-chave** que distinguem times de alta performance:

Métrica	O que mede
Deployment Frequency	Com que frequência o time coloca código em produção.
Lead Time for Changes	Tempo do commit até estar rodando em produção.
Change Failure Rate	% de deploys que causam falha em produção.
Time to Restore Service (MTTR)	Tempo para recuperar o serviço após um incidente.

O achado central do estudo: **velocidade e estabilidade não são um trade-off** — os times que fazem deploy com mais frequência também têm menos falhas e se recuperam mais rápido.

SLI, SLO, SLA e Error Budget

A prática de **Site Reliability Engineering (SRE)**, criada no Google e formalizada no *Site Reliability Engineering Book* (Beyer et al., 2016), trouxe seu próprio vocabulário de métricas para medir confiabilidade **em produção**, não apenas em teste:

- **SLI** (*Service Level Indicator*): a métrica medida de fato — ex.: % de requisições respondidas em menos de 200ms.
- **SLO** (*Service Level Objective*): a meta interna para o SLI — ex.: 99,9% das requisições em menos de 200ms, por mês.
- **SLA** (*Service Level Agreement*): o compromisso contratual com o cliente, geralmente mais frouxo que o SLO, com penalidades se descumprido.
- **Error Budget**: quanto de “falha” é tolerável antes de violar o SLO (ex.: 0,1% de erro/mês) — usado para decidir, com dados, se o time deve priorizar novas features ou estabilidade.

A Lei de Goodhart

Lei de Goodhart

“Quando uma medida se torna um alvo, ela deixa de ser uma boa medida.” — atribuída ao economista Charles Goodhart (1975), popularizada pela antropóloga Marilyn Strathern (1997).

Toda métrica é uma **proxy** — uma aproximação do que realmente queremos saber. Quando pessoas são avaliadas ou recompensadas diretamente por uma proxy, elas otimizam a proxy, não necessariamente o objetivo real por trás dela.

Conceitos relacionados: **Lei de Campbell** (avaliação por indicadores tende a corromper o processo que deveriam medir) e **Falácia de McNamara** (confiar só no que é facilmente quantificável, ignorando o resto).

Exemplos de métricas “jogadas” em software

- **Linhas de código como produtividade:** incentiva código verboso. Bill Gates já resumia: *“medir o progresso da programação em linhas de código é como medir o progresso na construção de um avião pelo peso”*.
- **Cobertura de testes de 100%:** times escrevem testes sem *asserts* relevantes só para bater a métrica, sem checar comportamento de fato.
- **Contagem de bugs fechados como meta:** incentiva fechar como “não é bug” ou “duplicado” em vez de corrigir.
- **Story points como “produtividade”:** incentiva inflação de estimativas ao longo do tempo (o mesmo trabalho “vale” cada vez mais pontos).

Discussão em sala

Pensem em uma métrica que vocês já viram (em estágio, projeto de faculdade ou notícia) sendo usada como meta direta. Como ela poderia ser “jogada” sem que o objetivo real fosse alcançado?

Avaliação de Métodos e Ferramentas

A literatura está repleta de descrições de novas ferramentas e métodos que prometem aumentar a produtividade das organizações e melhorar a qualidade e o custo de seus produtos. No entanto, é difícil distinguir promessas da realidade. Por isso, muitas organizações realizam experimentos, estudos de caso ou pesquisas para avaliar se determinada abordagem trará benefícios reais para seu contexto específico. Essas avaliações só são eficazes quando há medições e análises cuidadosas e controladas.

Métrica, Indicador e KPI: Entenda a Diferença



Figura 15: Dado, Métrica e Indicador

Medir a IA em vez de confiar nela

A IA generativa é hoje o exemplo mais atual de “ferramenta disruptiva aceita pela promessa”, do tipo que discutimos no início desta aula. É exatamente aqui que a avaliação de métodos e ferramentas se torna urgente: assistentes de código realmente aumentam a produtividade?

Um estudo de referência recente ajuda a responder — com dados, não com opinião.

Estudo de caso: METR (2025)

O *METR* (*Model Evaluation & Threat Research*) conduziu um **experimento controlado randomizado** com 16 desenvolvedores experientes (em média 5 anos no próprio repositório), resolvendo 246 tarefas reais em projetos open-source maduros, usando ferramentas como Cursor Pro e Claude 3.5/3.7 Sonnet.

Medida	Resultado
Previsão dos devs antes da tarefa	+24% mais rápido
Percepção dos devs depois da tarefa	+20% mais rápido
Tempo real medido	-19% (mais lento)

METR, *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity*, jul. 2025. arXiv:2507.09089. O próprio METR classifica o resultado como específico daquele contexto — pode não se repetir com modelos mais novos.

Por que isso importa para esta disciplina

- É um exemplo real de **divergência entre percepção e medição** — o tema central da nossa primeira aula sobre relevância da medição.
- Mostra por que **experimentos controlados** (tema central desta disciplina) são necessários mesmo quando “todo mundo sabe” que uma ferramenta funciona.
- É um lembrete de que resultados empíricos em software **envelhecem rápido** — a métrica certa hoje pode precisar ser remedida amanhã.

Métricas para avaliar modelos de IA

Além de medir se a IA ajuda *quem programa*, também precisamos medir a qualidade do *modelo em si* — que passa a ser mais um componente de software a ser avaliado:

- **Precision, Recall e F1-score:** acerto e cobertura de um classificador.
- **Taxa de alucinação:** frequência com que o modelo gera informação falsa apresentada como fato.
- **Benchmarks de código** (ex.: HumanEval, SWE-bench): % de tarefas de programação resolvidas corretamente.
- **Custo e latência por requisição:** cada resposta tem custo computacional e financeiro mensurável.

Essas métricas fogem do escopo desta disciplina, mas usam exatamente a mesma lógica do GQM: nenhuma delas faz sentido fora de um objetivo e uma pergunta bem definidos.

Recomendações de Medição



Figura 16: Recomendações de Medição

Atividade em sala

Em grupos de 3 a 4 pessoas, escolham um sistema que todos conheçam (o próprio sistema acadêmico da PUC, um app de entrega, uma rede social) e façam, em 15 minutos:

- 1 Escrevam **um objetivo** no template GQM (Analisar / com o propósito de / com respeito a / do ponto de vista de / no contexto de).
- 2 Derivem **2 ou 3 perguntas** que indicariam se esse objetivo está sendo alcançado.
- 3 Para cada pergunta, proponham **uma métrica** concreta.
- 4 Por fim, para cada métrica proposta: **como ela poderia ser “jogada”** (Lei de Goodhart) se virasse meta de alguém na empresa?

Um grupo apresenta o resultado; a turma tenta encontrar uma forma de burlar a métrica proposta antes do grupo revelar a própria resposta.

Sumário

- 1 Medição
- 2 Escopo das Métricas de Software
- 3 Métricas Modernas: Ágil, DevOps e IA
- 4 Normas Internacionais**
- 5 Artigos Recomendados

IEEE 1061 — Standard for a Software Quality Metrics Methodology

<https://standards.ieee.org/ieee/1061/1549/>

Propósito: Fornecer uma metodologia para identificar, definir e aplicar métricas de qualidade de software. Ele não especifica métricas fixas, mas descreve como criar e avaliar métricas adequadas para um determinado contexto ou projeto.

Enfoque: Ajuda a ligar atributos de qualidade (como confiabilidade, manutenibilidade, eficiência, usabilidade) a métricas mensuráveis.

IEEE 1061 — Standard for a Software Quality Metrics Methodology

Importância:

- Ajuda a transformar conceitos subjetivos (“o sistema é confiável”) em dados objetivos.
- Permite rastrear a melhoria ou degradação da qualidade ao longo do tempo.
- Foi um dos primeiros esforços de padronização em qualidade de software baseada em métricas, oferecendo uma estrutura neutra e reutilizável.
- Garantir comparabilidade entre projetos.
- Apoiar atividades de avaliação de processos e produtos de software.

IEEE 1061 — Standard for a Software Quality Metrics Methodology

Estrutura básica:

- 1 Identificar Atributos de qualidade (ex.: confiabilidade, usabilidade, manutenibilidade, eficiência, modularidade, acoplamento).
- 2 Escolher métricas para medir os atributos (ex.: número de defeitos/KLOC, tempo médio para reparo).
- 3 Validar se as métricas realmente medem o que se propõem.
- 4 Interpretação e uso: Como analisar resultados e aplicá-los em decisões de gestão de qualidade e engenharia de software.

IEEE 1061 — Standard for a Software Quality Metrics Methodology

Complementa o IEEE 730 (Software Quality Assurance Plans), que trata do planejamento da garantia da qualidade.

É relacionado à família ISO/IEC 9126 — hoje **substituída pela ISO/IEC 25010** (vista na seção *Escopo das Métricas de Software*), que define o modelo de qualidade de software atualmente em vigor.

Nota sobre a versão: a versão vigente é a **IEEE 1061-1998** (revisão de 1992), reafirmada em 2009 — daí a confusão comum com uma suposta “versão 2009”. Desde 2020, a norma está classificada como *Inactive-Reserved* pelo IEEE: não é mais atualizada, mas segue sendo referência histórica, tendo influenciado metodologias modernas de métricas em frameworks ágeis e em modelos de maturidade (CMMI, ISO/IEC 15939).

ISO/IEC/IEEE 15939 — Software Measurement Process

(edição vigente: ISO/IEC/IEEE 15939:2017 — ainda ativa, diferente da IEEE 1061)

Propósito: Definir um processo padronizado para a medição no ciclo de vida do software.

Enfoque: Mais amplo — não só qualidade, mas qualquer tipo de medição (custo, esforço, desempenho, defeitos, etc.).

Importância:

- Garante que a medição seja sistemática e repetível.
- Evita métricas isoladas sem ligação a objetivos estratégicos.
- Apoia modelos como CMMI, SPICE e ISO/IEC 12207.

ISO/IEC/IEEE 15939 — Software Measurement Process

Etapas Principais:

- 1 Planejar a medição: Definir objetivos, contexto e stakeholders.
- 2 Especificar métricas: Determinar quais dados serão coletados e como.
- 3 Coletar dados: Implementar mecanismos confiáveis e consistentes de coleta.
- 4 Analisar e interpretar: Gerar informações úteis para tomada de decisão.
- 5 Armazenar e manter: Preservar dados e resultados para uso futuro.

ISO/IEC 25012 — Modelo de Qualidade de Dados

Voltando ao ponto sobre *Coleta de Dados*: se a medição depende de dados confiáveis, como avaliar a qualidade dos próprios dados? A **ISO/IEC 25012** (2008, parte da família SQuaRE) define **15 características**, divididas em duas dimensões:

- **Inerentes aos dados:** acurácia, completude, consistência, credibilidade, atualidade (*currentness*).
- **Dependentes do sistema:** acessibilidade, conformidade, confidencialidade, eficiência, precisão, rastreabilidade, compreensibilidade, disponibilidade, portabilidade, recuperabilidade.

Relevância atual: é a base conceitual para praticamente toda iniciativa de *governança de dados* e para avaliar a qualidade dos dados usados em treinamento de modelos de IA — “garbage in, garbage out” também vale para medição de software.

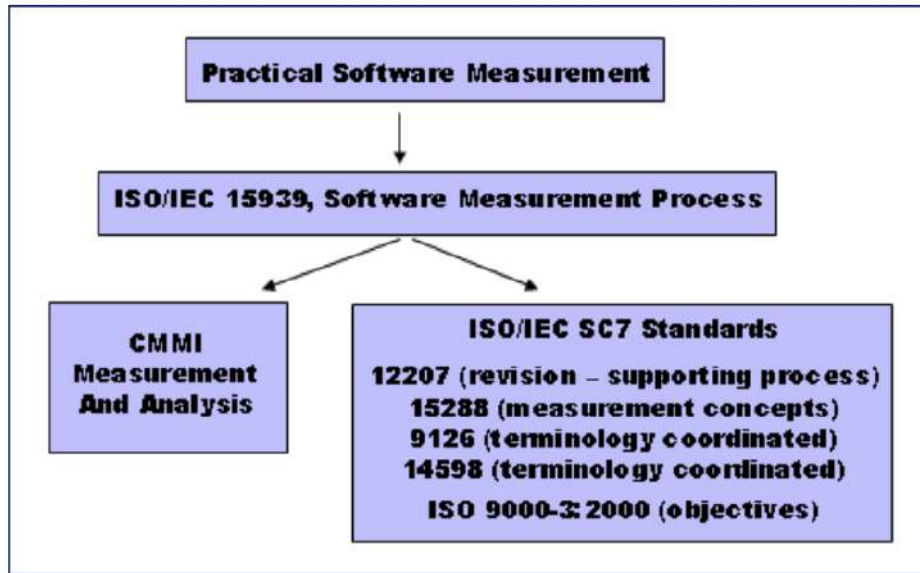


Figura 17: <https://www.psmisc.com/iso.asp>

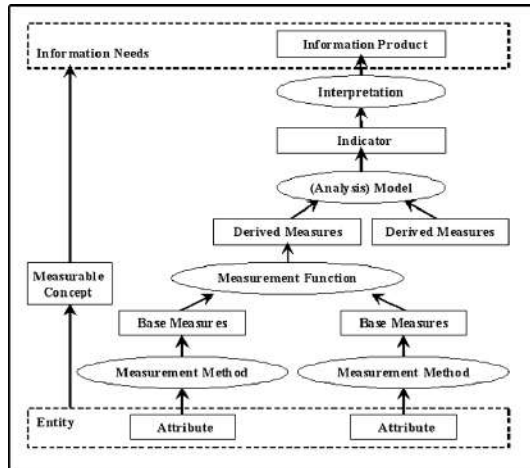


Figura 18: ISO 15939 measurement model.

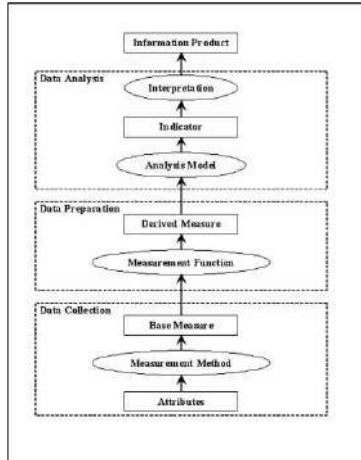


Figura 19: Data steps.

Sumário

- 1 Medição
- 2 Escopo das Métricas de Software
- 3 Métricas Modernas: Ágil, DevOps e IA
- 4 Normas Internacionais
- 5 Artigos Recomendados**

Artigos Recomendados

- https://nemo.inf.ufes.br/wp-content/papercite-data/pdf/recmed_uma_ferramenta_de_apoio_ao_uso_de_um_conjunto_de_recomendacoes_para_medicao_de_software_adequada_ao_controle_estatistico_de_processos_2012.pdf
- <http://www.din.uem.br/sbpo/sbpo2006/pdf/arq0222.pdf>

Leituras Modernas Recomendadas

- 1 BASILI, V.; CALDIERA, G.; ROMBACH, H. D. *The Goal Question Metric Approach*. University of Maryland / NASA-SEL, 1994.
- 2 BASILI, V. et al. *Aligning Organizations Through Measurement: The GQM+Strategies Approach*. Springer, 2014.
- 3 FORSGREN, Nicole; HUMBLE, Jez; KIM, Gene. *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.
- 4 BEYER, Betsy et al. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly, 2016. Disponível gratuitamente em: <https://sre.google/books/>.
- 5 METR. *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity*. jul. 2025. Disponível em: <https://arxiv.org/abs/2507.09089>.
- 6 ISO/IEC 25010:2023 — *Systems and software engineering — SQuaRE — Product quality model*.
- 7 DORA. *State of DevOps Report* (anual). Disponível em: <https://dora.dev>.

Bibliografia Básica

- 1 FENTON, Norman; BIEMAN, James. Software metrics: a rigorous and practical approach. CRC press, 2014.
- 2 WOHLIN, Claes et al. Experimentation in software engineering. Berlin: Springer, 2012.
- 3 JURISTO, Natalia; MORENO, Ana M. Basics of software engineering experimentation. Springer Science & Business Media, 2013.
- 4 EMPIRICAL SOFTWARE ENGINEERING: an international journal. Boston: Kluwer Academic Publishers, 1996-. j20. ISSN 1573-7616. Disponível em: <https://link.springer.com/journal/10664>. Acesso em: 22 jul. 2024. (Periódico On-line).
- 5 PRESSMAN, Roger S.; MAXIM, Bruce R.. Engenharia de software: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021. E-book. ISBN 9786558040118. (Livro Eletrônico).
- 6 WOHLIN, Claes et al. Experimentation in software engineering. New York: Springer, c2012. xxiii, 236 p. ISBN 9783642432262. (Disponível no Acervo).

Bibliografia Complementar

- 1 JURISTO, Natalia. Basics of Software Engineering Experimentation. 1st ed. 2001. XXII, 396 p ISBN 9781475733044. (Livro Eletrônico).
- 2 LAIRD, Linda M.,; BRENNAN, M. Carol,. Software measurement and estimation: a practical approach. Hoboken, N.J.: Wiley-Interscience, 2006. 1 online resource (xvi, 257 ISBN 0471792535 (electronic bk.) Disponível em : <https://ieeexplore.ieee.org/book/5988896>. Acesso em: 22 jul. 2024. (Livro Eletrônico).
- 3 SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson Education do Brasil, c2019. xii, 756 p. ISBN 9788543024974. (Disponível no Acervo).
- 4 DEVORE, Jay L. Probabilidade e estatística para engenharia e ciências. 3. São Paulo Cengage Learning 2018 1 recurso online ISBN 9788522128044. (Livro Eletrônico).
- 5 IEEE TRANSACTIONS ON SOFTWARE ENGINEERING. New York: IEEE Computer Society ,1975-. Mensal,. ISSN 0098-5589. Disponível em: <https://ieeexplore.ieee.org/xpl/RecentIssue.jsp?punumber=32>. Acesso em: 22 jul. 2024. (Periódico On-line).
- 6 JONES, Capers. Applied Software Measurement, 3rd edition. 2008. 1 online resource (696 pages). (Livro Eletrônico).